PROGRAMACIÓN ORIENTADA A OBJETOS · PYTHON

Fundamentos de la POO en Python

Clases, objetos, atributos, métodos y comportamiento

POO y programación procedural

Enfoque procedural

- Datos y funciones separados
- El estado circula entre funciones
- Exige más coordinación manual

Enfoque orientado a objetos

- Datos y conducta juntos
- Cada objeto conserva su estado
- Las clases definen reglas comunes

Enfoque Procedural

Datos y funciones están separados.

```
def crear_estudiante(nombre):


def calcular_promedio(estudiante):
```

Enfoque Orientado a Objetos (POO)

Datos y comportamiento se encapsulan en una única entidad.

```
class Estudiante:
    def promedio(self):
```

La POO proporciona mayor organización, mantenibilidad y escalabilidad para sistemas complejos.

Clases y objetos

- Una clase define la estructura y el comportamiento
- Un objeto es una instancia concreta de una clase
- Cada instancia tiene identidad propia
- La convención para clases es PascalCase

PascalCase

por convenio, los nombres de las clases en Python se escriben con la primera letra de cada palabra en mayúscula, sin espacios ni guiones bajos.

```
class Pato:
  """ significa literalmente: "no hagas nada"."""
  pass


class MonstruoGalletas:
  pass


class MascotaVirtual:
  pass


class CuentaBancaria:
  pass
```

Clase: El plano o plantilla que define características y comportamientos.

Objeto (Instancia): La entidad concreta creada a partir de la clase en memoria.

Identidad: Cada objeto ocupa un espacio único verificable con `type()` o `isinstance()`.

```
class Vehiculo:
    pass

mi_coche = Vehiculo()  # Instancia 1
tu_moto = Vehiculo()   # Instancia 2

print(mi_coche is tu_moto)  # False
```

Comportamiento

Los métodos de instancia definen las acciones que el objeto puede realizar.

Estado Interno

Los métodos operan de forma segura sobre los atributos del objeto usando self.

```
class Cuenta:
    def depositar(self, cantidad):
        self.saldo += cantidad
        return self.saldo

mi_cuenta = Cuenta()
mi_cuenta.depositar(500) # Modifica el estado
```

Primera clase en Python

```
class Vehiculo:
    pass

coche = Vehiculo()
moto = Vehiculo()

print(coche is moto)  # False
```

Dos objetos de la misma clase ocupan identidades distintas.

Constructor y atributos

```
class Vehiculo:
   def __init__(self, marca, modelo):
      self.marca = marca
      self.modelo = modelo
      self.encendido = False

coche = Vehiculo("Toyota", "Corolla")

“”” print(coche.marca, coche.modelo, coche.encendido, sep=" - ") sep="\n"”””

print(coche.marca, coche.modelo, coche.encendido)

#print(f"Marca: {coche.marca}")
# print(f"Modelo: {coche.modelo}")
# print(f"Encendido: {coche.encendido}")
```

__init__ establece un estado válido desde la creación del objeto.

El papel de self

- Referencia la instancia que invoca el método
- Permite leer y modificar sus atributos
- Debe aparecer como primer parámetro del método de instancia
- Python lo pasa automáticamente al llamar objeto.metodo()

Método inicializador. Se ejecuta automáticamente al instanciar (crear) el objeto.

Referencia a la instancia. Obligatorio como primer parámetro en métodos de instancia.

```
class Vehiculo:
    def __init__(self, marca):
        self.marca = marca
```

Atributo de instancia. Guarda el estado o los datos del objeto concreto.

Atributo de Clase

Compartido por todas las instancias.
Definido directamente dentro de la clase.

```
class Vehiculo:
    contador = 0  # Atributo de clase
```

Acceso: `Vehiculo.contador`

Atributo de Instancia

Único para cada objeto individual.
Definido dentro de __init__.

```
  def __init__(self, marca):
      self.marca = marca  # Atributo de instancia
```

Acceso: `mi_coche.marca`

Atributos de instancia y de clase

Instancia

- Se crean normalmente con self
- Pertenecen a un objeto
- Ejemplo: marca, saldo

Clase

- Se declaran en el cuerpo de la clase
- Se comparten entre instancias
- Ejemplo: contador, iva

Métodos que modifican estado

```
class Cuenta:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, cantidad):
        if cantidad <= 0:
            raise ValueError("Cantidad no válida")
        self.saldo += cantidad
        return self.saldo
```

Un método combina reglas del dominio con cambios controlados en el objeto.

Métodos de instancia, clase y estáticos

- Instancia: trabajan con self y el estado del objeto
- Clase: usan @classmethod y reciben cls
- Estáticos: usan @staticmethod y no reciben self ni cls
- La elección depende de los datos que necesita la operación

6 min

Representación con __str__ y __repr__

__str__ y __repr__ son métodos especiales que indican cómo debe representarse un objeto como texto.

```
class Libro:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor

    def __str__(self):
        return f'"{self.titulo}" por {self.autor}'

    def __repr__(self):
        return f"Libro({self.titulo!r}, {self.autor!r})"
```

__str__ prioriza la lectura humana. __repr__ ayuda a desarrollar y depurar.

```
def __str__(self):
    return f'"{self.titulo}" por {self.autor}'
```

Por ejemplo:

```
libro = Libro("El Quijote", "Miguel de Cervantes")

print(libro)
```

```
Resultado:  "El Quijote" por Miguel de Cervantes
```

__repr__: texto para el programador

Python llama a __repr__ cuando utilizamos repr():

```
def __repr__(self):
    return f"Libro({self.titulo!r}, {self.autor!r})“

print(repr(libro))
```

resultado

```
Libro('El Quijote', 'Miguel de Cervantes')
```


6 min

Diferencia principal

```
print(libro)      # Utiliza __str__
print(str(libro))   # Utiliza __str__

print(repr(libro))  # Utiliza __repr__
print([libro])     # La lista utiliza __repr__
```

En resumen:

- __str__: representación amigable para el usuario.
- __repr__: representación detallada para programadores y depuración.
- !r: aplica repr() al valor y, en una cadena, muestra sus comillas.

Método	Propósito	Ejemplo de Retorno
`__str__`	Representación legible para usuarios (`print`)	`f'{self.marca} {self.modelo}'`
`__eq__`	Define la comparación de contenido (==)	`self.marca == otro.marca`
`__add__`	Define comportamiento matemático (+)	`Vector2D(self.x + otro.x)`
`__len__`	Comportamiento como contenedor (`len()`)	`len(self.productos)`

__str__: cómo se muestra un objeto

QUÉ HACE

Devuelve un texto fácil de leer para una persona.

CUÁNDO SE EJECUTA

Al usar print(objeto) o str(objeto).

```
class Pato:
  def __init__(self, nombre):
    self.nombre = nombre

  def __str__(self):
    return f"Pato: {self.nombre}"

paco = Pato("Paco")
print(paco)
```

Resultado

Pato: Paco

Python llama al método automáticamente.

__eq__: cuándo dos objetos son iguales

QUÉ HACE

Define qué atributos deben coincidir para considerar iguales dos objetos.

CUÁNDO SE EJECUTA

Al utilizar objeto1 == objeto2.

```
class Pato:
  def __init__(self, nombre):
    self.nombre = nombre

  def __eq__(self, otro):
    return self.nombre == otro.nombre

a = Pato("Paco")
b = Pato("Paco")
print(a == b)
```

Resultado

True

Python llama al método automáticamente.

__add__: qué significa utilizar +

QUÉ HACE

Define el resultado de sumar dos objetos de nuestra clase.

CUÁNDO SE EJECUTA

Al utilizar objeto1 + objeto2.

```
class Bolsa:
  def __init__(self, galletas):
    self.galletas = galletas

  def __add__(self, otra):
    return self.galletas + otra.galletas

a = Bolsa(3)
b = Bolsa(2)
print(a + b)
```

Resultado

5

Python llama al método automáticamente.

__len__: cuánto contiene un objeto

QUÉ HACE

Devuelve el número de elementos almacenados dentro del objeto.

CUÁNDO SE EJECUTA

Al utilizar len(objeto).

```
class Caja:
  def __init__(self):
    self.objetos = []

  def guardar(self, objeto):
    self.objetos.append(objeto)

  def __len__(self):
    return len(self.objetos)

caja = Caja()
caja.guardar("pato")
print(len(caja))
```

Resultado

1

Python llama al método automáticamente.

Operadores y comparación

```
class Vector2D:
  def __init__(self, x, y):
    self.x, self.y = x, y

  def __add__(self, otro):
    return Vector2D(self.x + otro.x,
                    self.y + otro.y)

  def __eq__(self, otro):
    return isinstance(otro, Vector2D) and vars(self) == vars(otro)
```

isinstance() es una función de Python que comprueba si un objeto pertenece a una clase determinada

Los métodos especiales integran objetos propios con la sintaxis de Python.

6 min